

# ECE 310

## Lab 3 Report

Visnu Muthuraman

February 15, 2026

### Abstract

In this lab, a 4-bit Kogge–Stone Adder was designed using dataflow modeling in Verilog. The adder was built using propagate and generate signals along with a prefix network to compute carries in parallel. The goal of this lab was to understand how propagate and generate signals work together through multiple layers to form the carry signals. A testbench was used to apply different input combinations, including zero inputs, overflow cases, and cases that required full carry generation. The simulation results showed that the adder correctly produced the expected sum and carry-out values for all test cases.

## 1 Introduction, Background, and Theory

A binary adder is a circuit that adds two binary numbers and produces a sum and a carry-out. In fast adder designs, carry computation is very important because it determines the overall speed of the circuit.

The Kogge–Stone Adder is a type of parallel prefix adder that improves speed by computing carry signals in parallel rather than allowing them to ripple from one stage to the next. Instead of using full adders connected in series, the Kogge–Stone design uses propagate and generate signals. The propagate signal ( $P = A \oplus B$ ) indicates whether a carry will pass through a stage, and the generate signal ( $G = A \cdot B$ ) indicates whether a stage will produce a carry.

## 2 Design

The design was completed using data-flow modeling.

### 2.1 4-bit Kogge–Stone Adder

The 4-bit KSA calculates propagate and generate signals, uses prefix logic to compute the carries, and produces the final sum and carry outputs. Below is the implementation in Verilog.

```
1 module ksa_4bit_df(
2     input  [3:0] A,
3     input  [3:0] B,
4     output [3:0] S,
5     output Cout
6 );
7
8 wire [3:0] P0, G0, P1, G1, P2, G2;
9 wire [3:0] C;
10
11 assign P0 = A ^ B;
12 assign G0 = A & B;
13
14 assign P1[0] = P0[0];
15 assign G1[0] = G0[0];
16
17 assign P1[1] = P0[1] & P0[0];
18 assign G1[1] = G0[1] | (P0[1] & G0[0]);
```

```

19
20 assign P1[2] = P0[2] & P0[1];
21 assign G1[2] = G0[2] | (P0[2] & G0[1]);
22
23 assign P1[3] = P0[3] & P0[2];
24 assign G1[3] = G0[3] | (P0[3] & G0[2]);
25
26 assign P2[0] = P1[0];
27 assign G2[0] = G1[0];
28
29 assign P2[1] = P1[1];
30 assign G2[1] = G1[1];
31
32 assign P2[2] = P1[2] & P1[0];
33 assign G2[2] = G1[2] | (P1[2] & G1[0]);
34
35 assign P2[3] = P1[3] & P1[1];
36 assign G2[3] = G1[3] | (P1[3] & G1[1]);
37
38 assign C[0] = 1'b0;
39 assign C[1] = G2[0];
40 assign C[2] = G2[1];
41 assign C[3] = G2[2];
42 assign Cout = G2[3];
43
44 assign S = P0 ^ C;
45
46 endmodule

```

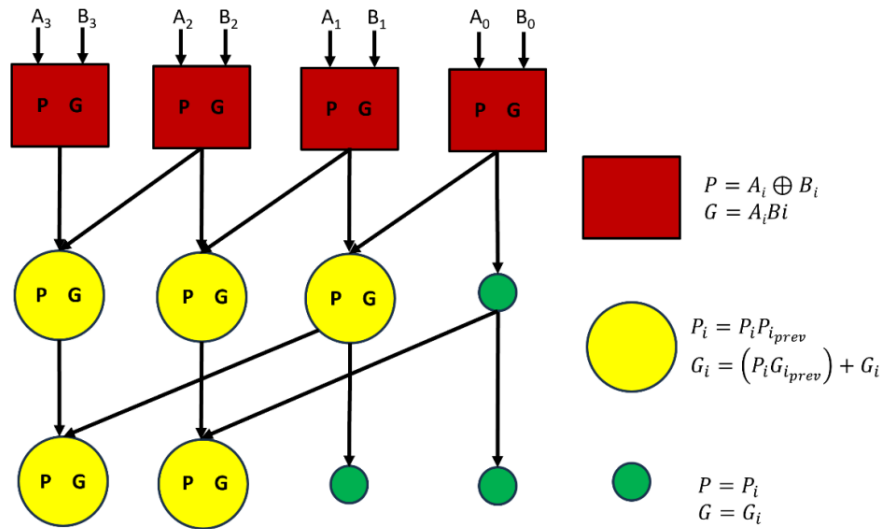


Figure 1: Diagram of 4-bit Kogge-Stone adder with layers.

## 2.2 Testbench

A testbench was written to check the behavior of the adder.

```

1 module ksa_4bit_df_tb();
2     reg [3:0] A, B;
3     wire [3:0] S;
4     wire Cout;
5
6     ksa_4bit_df dut(.A(A), .B(B), .S(S), .Cout(Cout));

```

```

7
8   initial begin
9       // Test 1: Base example (in pdf)
10      A = 4'b1010; B= 4'b1101;
11      #10;
12      $display("%b %b | %b %b", A, B, S, Cout);
13
14      // Test 2: No carry case
15      A = 4'b0011; B = 4'b0100;
16      #10;
17      $display("%b %b | %b %b", A, B, S, Cout);
18
19      // Test 3: Overflow case (final carry-out)
20      A = 4'b1111; B = 4'b0001;
21      #10;
22      $display("%b %b | %b %b", A, B, S, Cout);
23
24      // Test 4: Single carry generation
25      A = 4'b0001; B = 4'b0001;
26      #10;
27      $display("%b %b | %b %b", A, B, S, Cout);
28
29      $finish;
30   end
31
32 endmodule

```

These test cases were chosen to check normal operation, overflow, no carry, and single carry generation.

### 3 Results

The simulation results were observed using waveform output. The following cases were tested:

- Test 1: 1010 + 1101
- Test 2: 0011 + 0100
- Test 3: 1111 + 0001
- Test 4: 0001 + 0001

All outputs matched the expected binary results. Carry signals were computed correctly through the parallel prefix network, and the final sum and carry-out values were accurate for all test cases. No errors were observed during simulation.

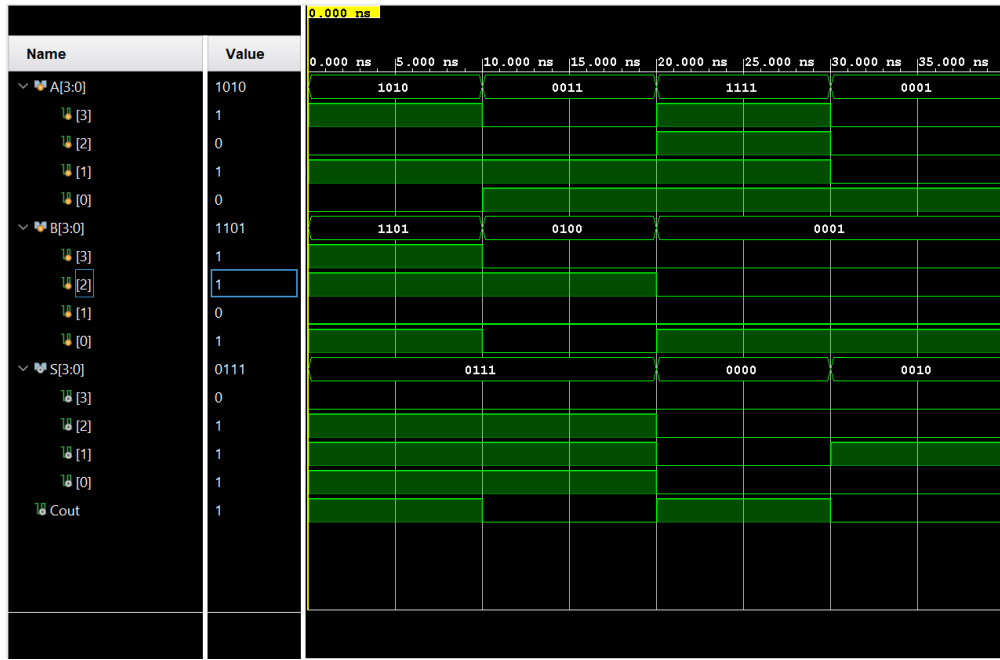


Figure 2: Waveform output of the 4-bit Kogge-Stone Adder simulation.

## 4 Conclusion

A 4-bit Kogge-Stone Adder was successfully implemented using dataflow modeling in Verilog. The design was tested using 4 input combinations, and correct results were observed in all cases. The lab demonstrated how carry signals are generated and combined through a parallel prefix structure and reinforced fundamental digital logic design concepts.

This report was compiled using L<sup>A</sup>T<sub>E</sub>X.